Jeudi 19/11, matin — Résumer un tableau : reduce et l'accumulateur

Kevin Bardoux

jeudi 19 novembre 2026

Table des matières

Objectifs 2
Matin — reduce et l'accumulateur 2
Rappel (5 min) 2
reduce (15 min) 2
reduce, en image 3
reduce, tour par tour 4
reduce, l'exemple minimal 4
reduce — attention et vérification 5
Pas à pas ensemble — Le total des visites (40 min) 5
Variante à faire seuls 8
Enregistrer et envoyer (2 min) 8
Exercices — reduce et l'accumulateur 8
Palier 1 — La somme (20 min) 8
Palier 2 — Le panier, le total des visites (45 min) 9
Palier 3 — Le mot le plus long, reduce à la main (40 min) 10
Enregistrer et envoyer (2 min) 11
À retenir 11
Si le temps le permet 11
Débrief 11
Sources 11

> Note
>
> **Compétences visées** : C4 · résolution de problème **Livrable** : le pas à pas `pas-a-pas-reduce` envoyé sur GitHub, et `node --test m3-somme-reduce.test.js` au vert.

> ! Le minimum du jour
>
> - Le pas à pas « Le total des visites », fait avec le groupe.
> - Le palier 1 : `node --test m3-somme-reduce.test.js` au vert.
>
> Les paliers 2 et 3, puis la rubrique « Si le temps le permet », sont la suite naturelle, pour ceux qui avancent.

Objectifs

- Résumer tout un tableau en une seule valeur avec `reduce`.
- Expliquer le rôle de l'accumulateur et de la valeur de départ.
- Suivre un `reduce` tour par tour, sur papier puis dans la console.

Comment se passe la matinée. Une seule notion : `reduce`. C'est la méthode la plus difficile de la semaine : nous la suivons tour par tour, au tableau, puis dans la console, avant de l'écrire.

Matin — reduce et l'accumulateur

Rappel (5 min)

1. Que donne `museum.Name` si la clé s'appelle `name` ?
2. `display: flex` : sur le parent, ou sur les enfants ?
3. Que fait `git pull` ?

Trois questions courtes, sans regarder vos notes : la veille (mardi), la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

reduce (15 min)

- **Pourquoi** : une seule valeur à partir de tout un tableau.
- **`reduce`** : un accumulateur, mis à jour à chaque élément.
- **Valeur de départ** : toujours donnée, en dernier argument.

Pourquoi. Le total des visites de tous les musées, le prix d'un panier, le mot le plus long : on veut **une** valeur, calculée à partir de

tout un tableau. `map` et `filter` renvoient des tableaux ; `reduce` renvoie ce que vous voulez : un nombre, un texte…

Définition. `reduce` exécute une fonction sur chaque élément du tableau en lui transmettant le résultat du tour précédent, l'**accumulateur**, pour aboutir à une seule valeur (MDN Web Docs, s. d.). Le dernier argument de `reduce` est la valeur de départ de l'accumulateur.

reduce, en image

`numbers.reduce((sum, n) => sum + n, 0)`, avec `numbers` qui vaut `[3, 8, 5]` :

Tour	`sum`	`n`	Renvoyé
départ	`0`	—	—
1	`0`	`3`	`3`
2	`3`	`8`	`11`
3	`11`	`5`	`16`
résultat			**`16`**

`sum` est l'accumulateur, `n` l'élément du tour, « Renvoyé » ce que la fonction renvoie. Au départ, `sum` vaut la valeur de départ, `0`. À chaque tour, il reçoit ce que la fonction a renvoyé au tour d'avant. Le résultat est **une seule valeur**, `16`. Le même calcul dans le visualiseur pas à pas : `sum` et `n` changent à chaque tour.

reduce, tour par tour

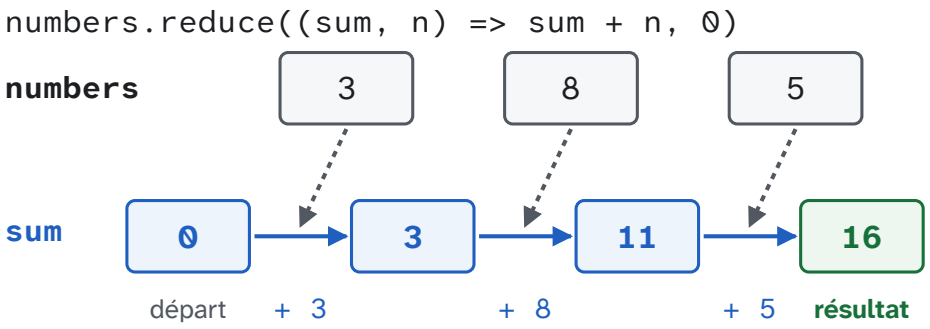


Figure 1. – À chaque tour, l'accumulateur sum reçoit ce que la fonction a renvoyé ; une seule valeur sort.

reduce, l'exemple minimal

Le premier programme de la notion, annoté :

```
const numbers = [3, 8, 5];

const total = numbers.reduce(
  (sum, n) => sum + n,
  0,
);

console.log(total); // 16
```

Ligne 3

`reduce` reçoit deux arguments : une fonction, puis une valeur de départ.

Ligne 4

La fonction reçoit l'accumulateur, `sum`, et l'élément du tour, `n` ; ce qu'elle renvoie devient le nouvel accumulateur.

Ligne 5

La valeur de départ de l'accumulateur : `0` pour une somme.

Ligne 8

Une seule valeur : l'accumulateur après le dernier tour.

reduce — attention et vérification

> ⚠ Attention
>
> Sans valeur de départ, `reduce` sur un tableau vide provoque une erreur : `TypeError: Reduce of empty array with no initial value`. Donnez toujours la valeur de départ : `0` pour une somme, `''` pour un texte.

- **Vérifiez** : `[2, 4].reduce((sum, n) => sum + n, 10)` ?
- Tracez les tours sur papier, puis vérifiez dans le REPL.

Pas à pas ensemble — Le total des visites (40 min)

- Le formateur code, vous tapez en même temps, une étape à la fois.
- Code et commentaires : `//` au-dessus de chaque ligne.
- Dossier : `pas-a-pas-reduce`, fichier `total.js`.
- Perdu ? Dossier `rattrapage/reduce/`, une étape à la fois.

Palier 1 : guidé. Ouvrez `pas-a-pas-reduce/total.js` : les visites de trois musées des Ardennes en 2019 (Ministère de la Culture, s. d.). Dans le terminal : `cd pas-a-pas-reduce`. Avant l'étape 1, tracez sur votre feuille, comme au tableau, les trois tours de la somme.

Avant de commencer. Lancez `node total.js`, depuis `pas-a-pas-reduce`. **Vous devez voir** : les données de départ, sans erreur. Chaque étape ajoute ses lignes à la fin du fichier, et sa sortie à la fin de la console.

Étape 1 — Additionner. À la fin du fichier, additionnez les visites avec `reduce`, en partant de `0`.

Avant d'exécuter : que va valoir le total ?

```
// reduce : additionne tout, en partant de 0
const total = visits.reduce((sum, n) => sum + n, 0);
// Une seule valeur : le total
console.log(total);
```

Enregistrez, puis lancez `node total.js` dans le terminal, depuis `pas-a-pas-reduce`.

Vous devez voir, à la fin de la sortie :

```
84795
```

Perdu ? Copiez `rattrapage/reduce/etape-1/total.js` à la place de votre `total.js`, puis repartez avec le groupe.

Étape 2 — Voir chaque tour. Refaites le même `reduce`, avec des accolades et un `console.log` à chaque tour.

Avant d'exécuter : au deuxième tour, que vaudra `sum` ?

```
// Trace : affiche chaque tour de reduce
visits.reduce((sum, n) => {
  // sum : l'accumulateur, le résultat du tour d'avant
  console.log(sum, n);
  // Avec des accolades, il faut return
  return sum + n;
}, 0);
```

Enregistrez, puis lancez `node total.js` dans le terminal, depuis `pas-a-pas-reduce`.

Vous devez voir, à la fin de la sortie :

```
0 3808
3808 29212
33020 51775
```

Perdu ? Copiez `rattrapage/reduce/etape-2/total.js` à la place de votre `total.js`, puis repartez avec le groupe.

Étape 3 — Sur des objets. Créez le tableau `museums`, puis additionnez le champ `total` de chaque objet.

Avant d'exécuter : le total sera-t-il le même qu'à l'étape 1 ?

```
// Tableau d'objets : les mêmes musées, avec leur nom
const museums = [
  // Chaque élément : un objet { name, total }
  { name: 'musée du feutre', total: 3808 },
```

```
  { name: 'musée Arthur Rimbaud', total: 29212 },
  { name: "musée de l'Ardenne", total: 51775 },
];
// reduce : additionne le champ total de chaque objet
const all = museums.reduce((sum, item) => sum +
item.total, 0);
// Le même total qu'à l'étape 1
console.log(all);
```

Enregistrez, puis lancez `node total.js` dans le terminal, depuis `pas-a-pas-reduce`.

Vous devez voir, à la fin de la sortie :

```
84795
```

Perdu ? Copiez `rattrapage/reduce/etape-3/total.js` à la place de votre `total.js`, puis repartez avec le groupe.

Étape 4 — Le plus grand. Gardez, à chaque tour, le plus grand total vu jusque-là, avec `Math.max`.

Avant d'exécuter : quel nombre va sortir ?

```
// reduce : garde le plus grand total vu jusqu'ici
const best = museums.reduce(
  // max : l'accumulateur ; item : le musée du tour
  (max, item) => Math.max(max, item.total),
  // Valeur de départ de l'accumulateur
  0,
);
// Le plus grand nombre de visites
console.log(best);
```

Enregistrez, puis lancez `node total.js` dans le terminal, depuis `pas-a-pas-reduce`.

Vous devez voir, à la fin de la sortie :

```
51775
```

Perdu ? Copiez `rattrapage/reduce/etape-4/total.js` à la place de votre `total.js`, puis repartez avec le groupe.

Variante à faire seuls
Pour ceux qui ont validé l'étape avant les autres, dans le même fichier. Additionnez seulement les musées de plus de 10 000 visites : `filter`, puis `reduce`. Affichez le résultat. **Réussi si** : la console affiche `80987`.

Enregistrer et envoyer (2 min)
Depuis `~/dwwm/dwwm-tp/semaine-07` : `git status`, `git add pas-a-pas-reduce`, `git commit -m "Ajoute le pas à pas reduce"`, puis `git push`. **Réussi si** : `git status` affiche `rien à valider, la copie de travail est propre`.

Exercices — reduce et l'accumulateur

Palier 1 — La somme (20 min)

1. `m3-somme-reduce.js` : la somme d'un tableau, avec `reduce`.
2. Tracez d'abord un exemple sur papier.
3. **Réussi si** : `node --test m3-somme-reduce.test.js` au vert.

Palier 1 : guidé. Durée : 20 min.

1. Ouvrez `m3-somme-reduce.js`. `sum(numbers)` renvoie la somme des nombres.
2. Sur papier : tracez `[1, 2, 3]`, tour par tour, comme au tableau.
3. Écrivez `return numbers.reduce((total, n) => …, 0);`.
4. Lancez `node --test m3-somme-reduce.test.js`. Le test « vide » vérifie la valeur de départ.
5. Un commit.

Réussi si : `node --test m3-somme-reduce.test.js` affiche `ℹ fail 0`.

> **i** Aides
>
> - **Le cours** : Jeudi matin, « reduce » ; le pas à pas, étape 1.
> - **L'IA** : CS50.ai (palier IA 1) : demandez ce que vaut l'accumulateur à chaque tour d'un exemple, pas le code.
> - **MDN** : Array.prototype.reduce().

Palier 2 — Le panier, le total des visites (45 min)

1. `m4-panier.js` : prix × quantité, sur des objets.
2. `dc4-total-visites.js` : un total inconnu compte pour `0`.
3. **Réussi si** : leurs tests au vert.

Palier 2 : recommandé, semi-guidé. Durée : 45 min.

1. **`m4-panier`** : `cartTotal(items)` reçoit des objets `{ name, price, quantity }` et renvoie le total à payer. À chaque tour, ajoutez à l'accumulateur le prix de l'article multiplié par sa quantité. `node --test m4-panier.test.js`.
2. **`dc4-total-visites`** : `totalVisits(museums)` additionne le champ `total` des musées. Un `total` qui vaut `null`, ou qui manque, compte pour `0` : `museum.total ?? 0` (encadré « Une valeur par défaut », mardi). `node --test dc4-total-visites.test.js`.

Réussi si : `node --test m4-panier.test.js` et `node --test dc4-total-visites.test.js` affichent `ℹ fail 0`.

Le dernier test lit le vrai fichier, déjà nettoyé (`data/musees-grand-est-2019-propres.json`, 114 musées) : 2 690 959 entrées en 2019.

> **i** Aides
>
> - **Le cours** : Jeudi matin, « reduce » ; le pas à pas, étape 3 (des objets).
> - **L'IA** : CS50.ai (palier IA 1) : demandez ce que vaut `100 + null` en JavaScript, et pourquoi on écrit quand même `?? 0`, pas le code.
> - **MDN** : Array.prototype.reduce() ; Opérateur de coalescence des nuls (??).

Palier 3 — Le mot le plus long, reduce à la main (40 min)

- `r3-plus-long-reduce.js` : un accumulateur qui est un texte.
- `tp-my-reduce.js` : `myReduce`, sans la méthode `reduce`.
- **Réussi si** : leurs tests `.defi` au vert.

Palier 3 : autonome. Durée : 40 min.

1. **`r3-plus-long-reduce`** : `longest(words)` renvoie le mot le plus long (le premier en cas d'égalité), ou `''` pour un tableau vide. Choisissez bien la valeur de départ. `node --test r3-plus-long-reduce.defi.test.js`.
2. **`tp-my-reduce`** : `myReduce(array, callback, initialValue)` fait ce que fait `reduce`, sans l'utiliser. `node --test tp-my-reduce.defi.test.js`.

Réussi si : les deux commandes affichent `ℹ fail 0`.

> **i** Autonome
>
> Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

> **Aides**
>
> - **Le cours** : Jeudi matin, « reduce » ; le pas à pas, étape 4 (garder le meilleur).
> - **L'IA** : CS50.ai (palier IA 1) : demandez quelle valeur de départ choisir pour un texte, pas le code.
> - **MDN** : Array.prototype.reduce(), section *Description*.

Enregistrer et envoyer (2 min)

Même rituel : `git status`, `git add`, `git commit -m "…"`, `git push`, depuis `semaine-07`.

À retenir

> **À retenir**
>
> - **`reduce`** : `(accumulateur, élément) => nouvel accumulateur`, puis la valeur de départ.
> - **Une seule valeur** sort : le dernier accumulateur.
> - **Tracer les tours** sur papier, avant d'écrire.

Si le temps le permet

- Les paliers 2 et 3 non terminés.
- Tracer un `reduce` sur papier, puis vérifier.

Débrief

- Que vaut l'accumulateur au premier tour ?
- Pourquoi donner une valeur de départ ?

Sources

MDN Web Docs. s. d. « Array : méthode reduce() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce.

Ministère de la Culture. s. d. « Fréquentation des Musées de France (data.gouv.fr) ». Consulté le 26 septembre 2026. https://www.data.gouv.fr/datasets/frequentation-des-musees-de-france-1.